JCUIScaler 1.0 – Technical Specification

FINAL – version R1-09

1. Introduction

JCUIScaler is an automatic user-interface scaling library for Lazarus LCL, with unofficial source-level compatibility with Delphi VCL (section 12.3). The library uses a deterministic reference model: every recalculation is performed only relative to the original state of the form and its controls, recorded on the first ScalerOnShow call after ScalerInit. This rules out the accumulation of rounding errors over repeated scaling.

The library can be integrated manually — the developer adds three calls to the form's events — or by means of the TJCScaler component (section 12.2), which performs the same calls automatically once placed on the form. The core works on its own; the component is purely an integration layer.

2. Architecture

The library core is based on a structure of type object (TJCUIScaler). It does not use the class model as the basis of its operation. The architecture is optimised for simplicity, predictability and safe operation — the object lives on the stack and requires no explicit freeing.

2.1 System modules

The core code is split into four files; the three .inc files are included into JCUIScaler.pas with the {$I} directive and have access to all private fields of the object. The TJCScaler component, its IDE registration and the Lazarus packages are in separate files and do not change the core. File names must be kept exactly as given — on Linux and macOS letter case matters:

File	Contents
JCUIScaler.pas	Main unit: declaration of TJCUIScaler, public API, ScaleForm, ScaleControl, ScalerInit, ScalerOnShow, ScalerOnResize.
JCUIScaler.capture.inc	Implementation of CaptureOriginals – iterative DFS, building the reference model.
JCUIScaler.controls.inc	Implementation of ApplyControlExceptions – scaling exceptions for special components (grids, ListView, TreeView, StatusBar, Splitter).
JCUIScaler.find.inc	Find*OrigIndex helper functions: FindDBGridOrigIndex, FindStrGridOrigIndex, FindCtrlOrigIndex, FindListViewOrigIndex, FindStatusBarOrigIndex, FindTreeViewOrigIndex – record lookups in the reference model.
JCUIScalerComponent.pas	TJCScaler component (LCL) — integration layer calling the public methods of TJCUIScaler (section 12.2).
design/JCUIScalerReg.pas	Registration of TJCScaler on the component palette (page JCUI) together with the icons (jcuiscaler_icons.res).
packages/lazarus/*.lpk	jcuiscaler_rt (RunTime: core + component) and jcuiscaler_dsgn (DesignTime: registration). Sources are taken from the source

	directory without copies.

3. Reference model

The reference model is a set of dynamic arrays of records filled once during CaptureOriginals (on the first ScalerOnShow after ScalerInit, when the form has already been scaled by the LCL). It stores the initial state of the form and of every control and is the only data source used during scaling — never the current screen state.

Defined record types:

Record	Stored data
TCtrlOrig	Every control: Left, Top, Width, Height, Font.Size. Array CtrlOrigs – the basis of generic scaling.
TDBGridOrig	TDBGrid: column widths (ColWidths[]), original grid client width (OrigGridWidth).
TStringGridOrig	TStringGrid: ColWidths[], OrigGridWidth – as for TDBGridOrig.
TListViewOrig	TListView: column widths (ColWidths[]), OrigWidth, OrigFontSize.
TTreeViewOrig	TTreeView: OrigFontSize. Fields OrigItemHeight and OrigIndent reserved (currently 0) – see section 6.
TStatusBarOrig	TStatusBar: panel widths (PanelWidths[]), OrigHeight.
TGroupBoxOrig	TGroupBox: OrigFontSize, OrigCaptionOffset (currently 0, best-effort).
TSplitterOrig	TSplitter: OrigWidth, OrigHeight.
TBorderOrig	TWinControl controls with the Tag bit JCSCALER_BORDER_TAG ($10000000) set: original values of BorderSpacing.Left/Top/Right/Bottom. LCL only.

> **NOTE:** The reference model is filled once, on the first ScalerOnShow after ScalerInit. Controls created dynamically after that moment are not registered automatically. A manual re-initialisation is required (calling ScalerInit again, followed by ScalerOnShow).

4. CaptureOriginals – building the reference model

The CaptureOriginals procedure performs an iterative DFS traversal of the form's control tree using its own stack (TWinControlStack). It avoids classic recursion, which eliminates the risk of stack overflow with deeply nested structures.

Algorithm:

```
1. Initialise the stack (INITIAL_STACK_SIZE = 256 elements, grows dynamically).
2. Push the root: FForm.
3. Main loop (while Top >= 0):
   a. Pop a TWinControl from the stack.
```

```
   b. Detect and register special components
      (TDBGrid, TStringGrid, TListView, TTreeView,
       TStatusBar, TGroupBox, TSplitter, TBorderOrig).
   c. For every child control (Root.Controls[i]):
      – always register a TCtrlOrig record (Left/Top/Width/Height/FontSize),
      – if the child is a TWinControl: push it onto the stack.
4. Safety limit: the loop ends after MAX_CONTROL_COUNT = 8192 controls.
```

An important consequence of the architecture: TToolButton is excluded from scaling at the ScaleControl stage (an is TToolButton check before the record lookup), even if it is in the reference model. TSpeedButton is not excluded and is scaled like any other control.

5. Scaling algorithm

5.1 Computing the factors

Every ScalerOnResize call runs ScaleForm, which computes two independent factors:

```
overallScaleX := FForm.Width  / FormOriginalWidth;
overallScaleY := FForm.Height / FormOriginalHeight;
```

No scaling takes place when the form is smaller than:

```
MIN_FORM_WIDTH  = 200 px
MIN_FORM_HEIGHT = 200 px
```

5.2 ScaleControl – generic scaling

For every control the position and size are computed from the reference model (never from the current values):

```
newL := LocalRound(co.OrigLeft   * overallScaleX);
newT := LocalRound(co.OrigTop    * overallScaleY);
newW := LocalRound(co.OrigWidth  * overallScaleX);
newH := LocalRound(co.OrigHeight * overallScaleY);
ctrl.SetBounds(newL, newT, newW, newH);
```

For TPanel, BevelWidth and BorderWidth are saved before SetBounds and restored after scaling (when SkipBevelScaling = True) — this prevents bevels from visually growing.

After the main scaling the procedure recursively scales the children (TWinControl.Controls[]) and then calls ApplyControlExceptions for the special cases.

5.3 ApplyControlExceptions – special component exceptions

After generic scaling every control passes through ApplyControlExceptions, which applies dedicated algorithms to components with a complex internal structure. The BorderSpacing section runs first and does not exit the procedure, so it also covers components with dedicated handling, and grids, ListView and StatusBar compute their column widths after the final BorderSpacing is set:

Component	Algorithm
TDBGrid	Proportional scaling of column widths relative to the current grid client width. The last column is corrected so that the sum = totalWidth (minus

	GridLastWOffset).
TStringGrid	Same algorithm as TDBGrid, but via the ColWidths[] property.
TListView	Cumulative scaling: the right edge of each column is computed as LocalRound(cumOrig * scaleFactor). The last column fills the remaining client width.
TStatusBar	Cumulative scaling of panel widths (as for TListView).
TTreeView	Scaling of Indent (overallScaleX). DefaultItemHeight scaled only under {$IFDEF LCLGTK2}. The LCL TTreeView has no ItemHeight property (up to R1-03 the code referred to it and did not compile for GTK2).
TSplitter	Scales Width (alLeft/alRight) or Height (alTop/alBottom) according to the alignment.
BorderSpacing	TWinControl controls with the Tag bit JCSCALER_BORDER_TAG have BorderSpacing.Left/Top/Right/Bottom scaled relative to the original values. LCL only — in Delphi the bit is ignored.

6. Supported components

Generic (bounds) scaling covers all controls registered in CtrlOrigs. Special handling applies to:

TPageControl, TStringGrid, TDBGrid, TListView, TTreeView, TStatusBar, TSplitter, TBevel (optional exclusion), TNoteBook.

Components excluded from scaling: TToolButton – skipped in ScaleControl regardless of the reference model contents. TSpeedButton is not excluded (specification corrected in R1-04; the exclusion concerned only problems observed with TToolButton).

TGroupBox: registered in the reference model (OrigFontSize, OrigCaptionOffset), but has no active section in ApplyControlExceptions in version 1.0. Handling will be added together with the implementation of font scaling.

6.1 TTreeView limitations

Fields OrigItemHeight and OrigIndent in TTreeViewOrig are currently stored as 0 (an explicit RC1 design decision, marked in the code with the FS3 comment). Indent scaling is active. DefaultItemHeight is scaled only under GTK2; the behaviour of this property on other LCL widgetsets has not been verified. With values of 0 neither operation has any effect in RC1.2. The scenario is planned to be resolved in a future version.

7. Font scaling

Font scaling is not implemented in version 1.0. An implementation attempt revealed deep dependencies between resolutions, DPI and platforms (including differences between Windows versions) that prevent deterministic and predictable behaviour on all supported configurations. The decision was made to roll it back and defer this area beyond version 1.0.

The OrigFontSize field is registered in the reference model (TCtrlOrig, TListViewOrig, TTreeViewOrig, TGroupBoxOrig) and restored by RestoreInitialScale — the infrastructure is ready for a future implementation once the platform dependency problem is solved.

> **NOTE:** Font scaling is deliberately disabled in version 1.0. Status: to be considered in a future version once the cross-platform problems are solved.

8. Event handling and public methods

8.1 ScalerInit

```
procedure ScalerInit(AForm: TForm)
```

Initialises the library. Stores the form reference and sets the default property values (SkipBevelScaling = True, ResetOnShow = False). The reference model, including the original size and position of the form (FormOriginalWidth, FormOriginalHeight, FormOriginalLeft, FormOriginalTop), is built only on the first ScalerOnShow call.

Call from: FormCreate.

> **NOTE:** The SkipBevelScaling and ResetOnShow properties must be set after ScalerInit — ScalerInit overwrites them with the defaults. Defaults: SkipBevelScaling = True, ResetOnShow = False.

8.2 ScalerOnShow

```
procedure ScalerOnShow
```

On the first call after ScalerInit it builds the reference model (CaptureOriginals). On subsequent calls it calls RestoreInitialScale if ResetOnShow = True; otherwise it does nothing.

Call from: FormShow.

> **NOTE:** The call is mandatory. Without ScalerOnShow the reference model is not built and ScalerOnResize performs no scaling.

8.3 ScalerOnResize

```
procedure ScalerOnResize
```

The main scaling method – calls ScaleForm, which recomputes the geometry of the whole control tree.

Call from: FormResize.

8.4 RestoreInitialScale (public)

Public emergency mechanism: restores the form to its state at the moment the reference model was built. The method is called automatically by ScalerOnShow when ResetOnShow = True, and is available to application code as a radical reset in unusual situations that cannot be defined in advance (e.g. specific behaviour of a widgetset or of a third-party control).

Operation: before the model is built (before the first ScalerOnShow) the method does nothing; it also does nothing when the form already has its original size and position. Otherwise it sets the form to its original size and position (FormOriginalLeft/Top/Width/Height) and scales the positions and sizes of the controls registered in the reference model (CtrlOrigs) back proportionally by the inverse factors ScaleBackX/Y (brute force — decision F-01). Controls outside the model are not modified directly. Font.Size values are also restored from OrigFontSize (no effect in 1.0, preparation for future font scaling).

Effects visible to the user: the window changes its position and size. If the method is called right after leaving the maximized state, while the form still reports the maximized size, the form goes through a WindowState switch (wsMaximized → wsNormal).

Consistency with the model state: resizing the form triggers OnResize, and thus ScalerOnResize, which recomputes all model controls at scale 1. The state after the reset is identical to the state of the first show — confirmed on GTK2 (pixel-exact screenshot comparison) and on Windows (Static Analysis, F-22 closed).

> **NOTE:** RestoreInitialScale does not cover the Minimize/Restore and Maximize/Restore scenarios. Handling of these cases depends on the behaviour of the LCL widgetset and does not guarantee an OnShow call.

9. Public properties and constants

9.1 Properties

Property	Type	Default	Description
SkipBevelScaling	Boolean	True	When True: BevelWidth and BorderWidth of TPanel panels are restored to their original values after every scaling. Prevents frames from visually growing.
ResetOnShow	Boolean	False	When True: calling ScalerOnShow restores the form to its original size and position (via RestoreInitialScale).

9.2 Public constants

Constant	Value	Description
MIN_FORM_WIDTH	200	Minimum form width at which scaling is active.
MIN_FORM_HEIGHT	200	Minimum form height at which scaling is active.
JCSCALER_BORDER_TAG	$10000000	Bit to set in a control's Tag to enable automatic scaling of its BorderSpacing. Applies to TWinControl controls; LCL only (ignored in Delphi). Example: MyPanel.Tag := MyPanel.Tag or JCSCALER_BORDER_TAG;

9.3 LocalRound (private method)

All pixel computations in the library use the private method LocalRound instead of the built-in Round function. LocalRound implements Floor(V + 0.5) — deterministic rounding of *.5 values always upwards (towards +∞), independent of the rounding mode. Up to R1-03 Trunc(V + 0.5) was used, which gives the same result for positive values but rounds negative values incorrectly (e.g. -1.7 → -1 instead of -2; this affects, among others, controls in a scrolled TScrollBox). The built-in Round uses banker's rounding (round-half-to-even), which can give asymmetric pixel results with repeated scaling.

10. Design assumptions and limitations

Assumption	Details
Scaling relative to the reference model	Eliminates error accumulation – every recalculation starts from the original state.
No dependency on the system DPI	The library does not read the DPI – it scales only on the basis of the window geometry.
Core based on object	TJCUIScaler lives on the stack. No Create/Free required.
No modification of widgetsets or the LCL	The library does not change any LCL mechanisms.
Dynamic arrays for the model only	The record arrays are allocated once in CaptureOriginals (first ScalerOnShow) and do not grow during scaling.
No support for dynamic controls	Controls created after the model has been built are not registered automatically. ScalerInit and ScalerOnShow must be called again.
No font scaling (1.0)	Rolled back because of deep DPI/platform/OS version dependencies. To be considered in a future version.
TToolButton excluded	Scaling of TToolButton is explicitly skipped in ScaleControl. TSpeedButton is scaled normally.
TListView on a hidden tab sheet	A native TListView on a hidden tab sheet reports a ClientWidth without its vertical scrollbar. Columns scaled at that time are wider by the scrollbar width until the next form resize. Recommendation: ScrollBars = ssVertical (section 13.1). LCL grids are not affected.
Anchors = [] at a different DPI	Controls without anchors are moved by the LCL during automatic DPI scaling (DPI different from DesignTimePPI) before the reference model is built. Recommendation: default Anchors = [akLeft, akTop] (section 13.1).
TJCScaler — replacing OnShow at run time	The component chains the form's OnShow in Loaded (the LCL has no handler list for OnShow). If the application assigns a new OnShow at run time (including in FormCreate), Attach must be called afterwards.

11. Public API – summary

Signature	Description
procedure ScalerInit(AForm: TForm)	Initialisation – always first. Sets the defaults; the reference model is built on the first ScalerOnShow.
procedure ScalerOnShow	On the first call it builds the reference model, later it optionally restores the state (ResetOnShow). Call from FormShow — mandatory.
procedure ScalerOnResize	Scales the whole form. Call from FormResize.

procedure RestoreInitialScale	Public emergency mechanism — restores the state at the moment the reference model was built. Called by ScalerOnShow when ResetOnShow = True, or directly from application code. See 8.4.
property SkipBevelScaling: Boolean	Protects TPanel bevels. Default True.
property ResetOnShow: Boolean	Restores the original state on OnShow. Default False.
JCSCALER_BORDER_TAG = $10000000	Constant to set in a control's Tag — enables BorderSpacing scaling.

12. Integration with a form

12.1 Manual integration

Copy the library files (JCUIScaler.pas, JCUIScaler.capture.inc, JCUIScaler.controls.inc, JCUIScaler.find.inc) to the project directory, add the JCUIScaler unit to the uses clause and add a field FScaler: TJCUIScaler to the form class.

```
type
  TMainForm = class(TForm)
  private
    FScaler: TJCUIScaler;
  end;

procedure TMainForm.FormCreate(Sender: TObject);
begin
  FScaler.ScalerInit(Self);
  // Optionally, after ScalerInit (ScalerInit sets the defaults):
  // FScaler.SkipBevelScaling := True;   // default True
  // FScaler.ResetOnShow := False;        // default False
end;

procedure TMainForm.FormShow(Sender: TObject);
begin
  FScaler.ScalerOnShow;
end;

procedure TMainForm.FormResize(Sender: TObject);
begin
  FScaler.ScalerOnResize;
end;
```

NOTE: The order matters: ScalerInit in FormCreate, properties set after ScalerInit, ScalerOnShow in FormShow, ScalerOnResize in FormResize. Until the first ScalerOnShow, ScalerOnResize performs no scaling.

12.2 Component integration — TJCScaler

TJCScaler (unit JCUIScalerComponent) is an integration layer for the LCL: placed on a form, the component performs the three core calls automatically. It does not replace the core and does not change its behaviour — it holds a TJCUIScaler object and calls only its public methods. Installation: package packages/lazarus/jcuiscaler_dsgn.lpk (requires jcuiscaler_rt.lpk); the component appears on the JCUI palette page.

Class name: TJCScaler, so that the default instance name on a form is short (JCScaler1). The core object keeps the name TJCUIScaler — it is used in production with the manual integration and its name cannot change without breaking compatibility.

Operation: Loaded calls Attach — ScalerInit for the owner form, then the properties are applied. The form's OnShow is chained; on every show the application's OnShow is called first, then ScalerOnShow — in the same order as with the manual integration. Resizing is handled via AddHandlerOnResize (the LCL handler list); the application's OnResize stays untouched and is called first. The component's results are pixel-identical to the manual integration.

Properties (published, linked to the core object): SkipBevelScaling (default True) and ResetOnShow (default False). They can be set in the Object Inspector or changed in code at any time — after Attach they are read from and written to the TJCUIScaler object directly.

Events (published, empty by default; Sender = the component): BeforeShow, AfterShow, BeforeResize, AfterResize. Order when the form is shown: form OnShow → BeforeShow → ScalerOnShow → AfterShow. Order on resize: form OnResize → BeforeResize → ScalerOnResize → AfterResize. On the first show BeforeShow runs before the reference model is built. AfterResize is the place for adjustments the scaler does not make (fonts, controls outside the model, custom-drawn controls); the geometry of model controls set in BeforeResize is overwritten by the scaler.

Extending in a descendant class: protected virtual methods DoShow and DoResize (default: Before* event, scaler, After* event) and the protected property Attached. Public methods: Attach (for a component created in code or after replacing OnShow at run time), Rescale (= ScalerOnResize, e.g. in TPageControl.OnChange on Windows — section 13.1) and RestoreInitialScale.

Limitations: one component per form (the first one is active); the owner must be a TForm — otherwise (frame, data module) the component stays inactive; LCL only (uses LCL handler lists). The demo project deliberately uses the manual integration and needs no package installation.

12.3 Delphi (unofficial)

The JCUIScaler source code also compiles with Delphi VCL — confirmed with Delphi 13.2 (Win64): compilation and basic operation, including third-party controls (TMS UI Pack PageControl, RxDBGrid). The support is unofficial and limited to a manual integration identical to section 12.1: the same four files, the same uses clause, the same three calls. No Delphi package is provided — the .lpk package applies to Lazarus only.

The object-based architecture remains unchanged. Differences between FPC/LCL and Delphi/VCL are handled with neutral code (access class TJCControlAccess for the protected TControl.Font in VCL, explicit initialisation of local variables) and three {$IFDEF FPC} blocks: the compiler mode directive and the BorderSpacing capture and apply. VCL units are resolved through the standard unit scope names of a VCL project.

Limitations in Delphi: JCSCALER_BORDER_TAG is ignored (VCL has no BorderSpacing; Margins has different semantics and is not emulated); monitor DPI changes (R2-01) are not handled — VCL per-monitor scaling may interfere with the scaler in multi-monitor configurations; TJCUIScaler should be declared as a form field (as in section 12.1); in VCL TSplitter descends from TGraphicControl, so it receives only the generic bounds scaling (no dedicated handling from section 5.3).

The demo project works in Lazarus only: apart from the missing .dfm files it relies on LCL-specific elements (.lfm/.lpi files, the DateTimeCtrls package, LCL property names in the forms).

The TJCScaler component and the .lpk packages are available for Lazarus only; in Delphi the manual integration is used.

Source files use CRLF line endings (required by RAD Studio, accepted by Lazarus); a .gitattributes file keeps them in the repository.

13. Compatibility

Item	Requirement
Lazarus	3.0+ (tested: 3.0 — Linux/GTK2, 4.8 — Windows, 4.8 — macOS)
Free Pascal	3.2.2+
Operating systems	Windows, Linux, macOS
Widgetsets	Win32/Win64, GTK2 (DefaultItemHeight TreeView) – with the reservations described in section 6.1; Cocoa (macOS 27) – tested; Qt – untested
Delphi (unofficial)	Delphi 13.2 (Win64), VCL — verified: compilation and basic operation; manual integration only, no package. See section 12.3.

13.1 Control configuration recommendations

Anchors. Controls scaled by JCUIScaler should keep the default Anchors = [akLeft, akTop]. Clearing the anchors (Anchors = []) makes the LCL move controls during automatic DPI scaling when the DPI differs from the form's DesignTimePPI — before the first ScalerOnShow. The scaler then records the already broken layout in the reference model and faithfully preserves it. Verified on GTK2 at 96 and 120 DPI (DesignTimePPI = 120).

TListView scrollbars (vsReport): ScrollBars = ssVertical is recommended. JCUIScaler fits the columns to the control width, so a horizontal scrollbar is not needed. A native TListView on a hidden tab sheet reports a ClientWidth without its vertical scrollbar; columns scaled at that time are wider by the scrollbar width until the next form resize (Windows and GTK2; GTK2: 451 px on the hidden tab, 435 px once shown). With ssVertical the excess stays under the vertical scrollbar — only the empty end of the last column is clipped and no horizontal scrollbar appears. The solution requires no code and no extra calls.

LCL grids (TStringGrid, TDBGrid) do not have this problem: they are drawn by the LCL and manage their own scrollbars, so ClientWidth is correct also on a hidden tab sheet (GTK2: ssAutoBoth, ssVertical, ssBoth; 6 and 60 rows). ssVertical is a sensible default for them, but it is not required.

Optional, Windows only: exact column widths immediately after a tab change are obtained by calling FScaler.ScalerOnResize in TPageControl.OnChange (with the component: JCScaler1.Rescale). ScalerOnResize recomputes everything from the reference model, so the call is deterministic and safe at an unchanged form size. It does not work on GTK2: TTabSheet.OnShow and TPageControl.OnChange are called before the new tab is laid out.

GTK2: the default ScrollBars = ssBoth is mapped to GTK_POLICY_ALWAYS by the GTK2 widgetset — both scrollbars are always visible, regardless of content.

Rejected solutions (contrary to the library's design or unreliable): an OnResize handler registered by the scaler on controls (the scaler would react to events outside its three explicit calls and its lifetime would be tied to the controls); a deferred call (Application.QueueAsyncCall) — visible flicker; reserving the scrollbar width — on GTK2 GetSystemMetrics(SM_CXVSCROLL) = 16 px while the actual scrollbar width is 19 px.

13.2 License

JCUIScaler is distributed under the same license as the Lazarus LCL: the GNU Library General Public License, version 2 (file COPYING.LGPL.txt), with the static linking exception (file COPYING.modifiedLGPL.txt — exception text identical to Lazarus). The LICENSE, COPYRIGHT and AUTHORS files are in the root directory.

Every source file contains a header with the copyright notice (Copyright (c) 2025-2026 Jakub Cebula, Kraków) and a reference to the license file. The copyright notice and the original author's name must not be removed from the source files or from modified versions of them; this does not change the license terms.

14. Scope of technical tests

The tests verify the following scenarios:

- Resize (changing the window size in any direction)
- Maximize/Restore
- PageControl (tabs, switching pages)
- Grids (TStringGrid, TDBGrid – column scaling, last column)
- Nested structures (controls inside panels, GroupBox)
- Regression (no drift after repeated resizing)
- Compilation: FPC/LCL with all supported widgetsets (including LCLGTK2) and Delphi VCL (Delphi 13.2, Win64 — compilation, basic operation, third-party controls)
- Runtime tests: DPI different from DesignTimePPI (96 and 120 DPI), resizing with a hidden tab (TListView), second form with ResetOnShow and JCSCALER_BORDER_TAG
- TJCScaler component: pixel-identical results compared with the manual integration (component from the .lfm file and created in code), event order, properties linked to the object, virtual methods
- Platforms: Windows 11 (Lazarus 4.8), macOS 27 (Lazarus 4.8, Cocoa), Linux (Lazarus 3.0, GTK2); Delphi 13.2 (Win64, unofficial)

15. Future directions

Area	Description
Font scaling	To be considered after 1.0. Requires solving the DPI/platform/OS version dependencies. The infrastructure (OrigFontSize) is ready in the reference model.
TGroupBox in ApplyControlExceptions	Tied to font scaling — will be added together with it.
TreeView DefaultItemHeight	A cross-widgetset solution for scaling DefaultItemHeight beyond GTK2.
Re-initialisation after a monitor change	Handling of scenarios where the window is moved between monitors with different scaling.
Delphi — monitor DPI change	Handling via OnBeforeMonitorDpiChanged / OnAfterMonitorDpiChanged (VCL) following the R2-01 hypothesis.
OPM package	Distribution through the Online Package Manager.
PL/EN documentation	Polish version as the source document; English translation.

16. Conclusions from the RC1 static analysis

After the cleanup session of 2026-06-01 the RC1 code is architecturally and implementationally consistent. All findings of static analysis Rev D were resolved: some fixed in the code, some closed by deliberate design decisions with a documented rationale.

The reference model is complete and correctly filled by CaptureOriginals for all supported component types. The column scaling algorithms (DBGrid, StringGrid, ListView, StatusBar) are deterministic and resistant to the accumulation of rounding errors thanks to the cumulative method with last-column correction and the uniform LocalRound function.

RC1.2 (2026-11-06, Static Analysis Rev G) fixed three bugs not detected in revisions D–F: the control position stored instead of BorderSpacing in the reference model (together with the BorderSpacing section being unreachable for components with dedicated handling), failure to compile for LCLGTK2 (the non-existent ItemHeight property) and incorrect rounding of negative values in LocalRound. Unofficial source-level compatibility with Delphi VCL was also introduced. The specification was corrected regarding TSpeedButton (it is not excluded) and the moment the reference model is built (the first ScalerOnShow).

The runtime tests of 2026-11-07 (GTK2, 96 and 120 DPI; Static Analysis Rev H) confirmed no drift after resize cycles, correct BorderSpacing scaling and correct scaling of all demo tabs. They revealed two behaviours requiring configuration recommendations (section 13.1): Anchors = [] at a DPI different from DesignTimePPI, and TListView on a hidden tab sheet. The library code needed no changes.

RC2 (2026-11-07, Static Analysis Rev I) introduces the TJCScaler component, the Lazarus packages and the license files. The core is unchanged in code (changes in comments only: license headers, the RestoreInitialScale description). The component was checked against the critical architecture risks (Risk Analysis Rev C) — it is compliant.

Version 1.0: code, packages, license and the Polish and English documentation (with the README) are complete. The library is ready to be published in a public repository and submitted to OPM. Compilation and basic operation with Delphi 13.2 have been confirmed.

The demo was also verified on macOS 27 with Lazarus 4.8 (Cocoa). Tested platforms: Windows, macOS and Linux/GTK2, and — unofficially — Delphi 13.2.

17. Notes on the documentation

This English version is a translation of the Polish source document; in case of discrepancies the Polish version prevails. Documentation version numbers are independent of code version numbers.

The technical specification is the current, maintained document. The supplementary documents — Static Analysis, Risk Analysis and RoadMap — are in the Additional subdirectory; they describe the development process up to version 1.0 and are not updated on an ongoing basis. References to findings F-xx in this specification refer to the register in the Static Analysis.

The rules for contributing changes, including the architecture rules in force, are maintained in the CONTRIBUTING.md file. Further planning and new findings are tracked in the project repository (GitHub issues).